Placement Dataset

```
import pandas as pd
import numpy as np
```

```
df1=pd.read_csv("/content/final_dataset_1.csv")
df1
```

	Enrollment_ID	Name	Department	Email	Attendance	Mock_Status	Mock_Score	CGPA	Internship
0	70000093	Amit Kumar	IT	amit.kumar@gmail.com	42.0	Absent	NaN	7.20	4
1	70001863	Arjun Singh	CSIT	NaN	47.0	Absent	NaN	6.98	2
2	70001088	Amit Gupta	DS	NaN	87.0	Absent	NaN	NaN	4
3	70000292	Neha Patel	DS	neha_patel@outlook.com	52.0	Absent	NaN	8.75	1
4	70000752	Pooja Singh	AIML	NaN	95.0	Present	93.10	9.52	1
...	...	...	...	...	...	...	...	...	
18865	95001229	Neel Pandey	CSE	NaN	79.0	Absent	NaN	8.85	3
18866	95002169	Aditi Mittal	CSE	aditi.mittal@gmail.com	NaN	Absent	NaN	5.44	1
18867	95000277	Vikas Kulshreshtha	CSE	vikas_kulshreshtha@outlook.com	97.0	Absent	NaN	NaN	1
18868	95001809	Neel Choudhary	CSE	neel.choudhary@gmail.com	70.0	Present	71.73	9.14	3
18869	95000191	Snehal Garg	DS	snehal.garg@gmail.com	93.0	Present	89.73	7.67	1

18870 rows × 13 columns

Next steps: Generate code with df1 New interactive sheet

Start coding or generate with AI.

```
print("\n")
print("Shape :",df1.shape)
print("Info :  \n")
df1.info()
print("\n")
print("Describe:  \n")
df1.describe()
```

```
Shape : (18870, 13)
Info :

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 18870 entries, 0 to 18869
Data columns (total 13 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Enrollment_ID        18870 non-null  int64
 1   Name                 18870 non-null  object
 2   Department           18870 non-null  object
 3   Email                13609 non-null  object
 4   Attendance           15740 non-null  float64
 5   Mock_Status          18870 non-null  object
 6   Mock_Score           10834 non-null  float64
 7   CGPA                 16420 non-null  float64
 8   Internships          15629 non-null  float64
 9   Projects             15629 non-null  float64
 10  Aptitude_Score       15696 non-null  float64
 11  Communication_Score  15756 non-null  float64
 12  Placed               18870 non-null  object
dtypes: float64(7), int64(1), object(5)
memory usage: 1.9+ MB


Describe:
```

	Enrollment_ID	Attendance	Mock_Score	CGPA	Internships	Projects	Aptitude_Score	Communication_Score
count	1.887000e+04	15740.000000	10834.000000	16420.000000	15629.000000	15629.000000	15696.000000	15756.000000
mean	7.752263e+07	72.104193	53.884071	7.405868	1.820974	3.329580	62.732302	62.741976
std	1.055583e+07	16.512615	29.203613	1.339802	1.340226	1.629044	21.144506	21.045742
min	7.000000e+07	40.000000	0.000000	5.000000	0.000000	1.000000	20.000000	20.010000
25%	7.020045e+07	59.000000	35.560000	6.240000	1.000000	2.000000	46.280000	46.390000
50%	7.040090e+07	73.000000	57.600000	7.430000	2.000000	3.000000	63.285000	63.440000
75%	9.000143e+07	86.000000	77.447500	8.560000	3.000000	5.000000	80.740000	80.502500
max	9.500300e+07	99.000000	99.990000	9.800000	4.000000	6.000000	99.990000	99.990000

```
df1.isnull().sum()
```

	0
Enrollment_ID	0
Name	0
Department	0
Email	5261
Attendance	3130
Mock_Status	0
Mock_Score	8036
CGPA	2450
Internships	3241
Projects	3241
Aptitude_Score	3174
Communication_Score	3114
Placed	0

dtype: int64

Data Preprocessing /Data Cleaning

```
df1["Aptitude_Score"].fillna(df1["Aptitude_Score"].mean(),inplace=True) # all null values will have average value
```

```
/tmp/ipykernel_4752/2075202888.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chain
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df


  df1["Aptitude_Score"].fillna(df1["Aptitude_Score"].mean(),inplace=True) # all null values will have average value
```

```
df1["Communication_Score"].fillna(df1["Communication_Score"].mean(),inplace=True) #all null values in communication score will
```

```
/tmp/ipykernel_4752/3450942286.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chain
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df


  df1["Communication_Score"].fillna(df1["Communication_Score"].mean(),inplace=True) #all null values in communication score will
```

```
df1["CGPA"].fillna(df1["CGPA"].mean(),inplace=True) # it will add mean cgpa in all null values present in column of cgpa
```

```
/tmp/ipykernel_4752/301695744.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chaine
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df


  df1["CGPA"].fillna(df1["CGPA"].mean(),inplace=True) # it will add mean cgpa in all null values present in column of cgpa
```

```
df1["Internships"]=df1["Internships"].fillna(0) # it will fill all null values or missing values with zero
```

```
df1["Projects"]=df1["Projects"].fillna(1) # it will add one to all null values or missing values present in  project column.
```

```
df1["Attendance"]=df1["Attendance"].fillna(df1["Attendance"].median())# we use median to avoid outliers
```

Double-click (or enter) to edit

```
df1.loc[df1["Mock_Status"] == "Absent", "Mock_Score"] = 0
```

```
df1["Mock_Status"]=df1["Mock_Status"].map({"Present":1,"Absent":0})
```

```
df1["Placed"]=df1["Placed"].map({"Placed":1,"Not Placed":0})
```

```
df1
```

	Enrollment_ID	Name	Department	Email	Attendance	Mock_Status	Mock_Score	CGPA	Intern
0	70000093	Amit Kumar	IT	amit.kumar@gmail.com	42.0	0	0.00	7.200000	
1	70001863	Arjun Singh	CSIT	NaN	47.0	0	0.00	6.980000	
2	70001088	Amit Gupta	DS	NaN	87.0	0	0.00	7.405868	
3	70000292	Neha Patel	DS	neha_patel@outlook.com	52.0	0	0.00	8.750000	
4	70000752	Pooja Singh	AIML	NaN	95.0	1	93.10	9.520000	
...	...	...	...	...	...	...	...	...	
18865	95001229	Neel Pandey	CSE	NaN	79.0	0	0.00	8.850000	
18866	95002169	Aditi Mittal	CSE	aditi.mittal@gmail.com	73.0	0	0.00	5.440000	
18867	95000277	Vikas Kulshreshtha	CSE	vikas_kulshreshtha@outlook.com	97.0	0	0.00	7.405868	
18868	95001809	Neel Choudhary	CSE	neel.choudhary@gmail.com	70.0	1	71.73	9.140000	
18869	95000191	Snehal Garg	DS	snehal.garg@gmail.com	93.0	1	89.73	7.670000	

18870 rows × 13 columns

Next steps: Generate code with df1 New interactive sheet

```
df1.drop_duplicates() # we drop all duplicates row
```

	Enrollment_ID	Name	Department	Email	Attendance	Mock_Status	Mock_Score	CGPA	Intern
0	70000093	Amit Kumar	IT	amit.kumar@gmail.com	42.0	0	0.00	7.200000	
1	70001863	Arjun Singh	CSIT	NaN	47.0	0	0.00	6.980000	
2	70001088	Amit Gupta	DS	NaN	87.0	0	0.00	7.405868	
3	70000292	Neha Patel	DS	neha_patel@outlook.com	52.0	0	0.00	8.750000	
4	70000752	Pooja Singh	AIML	NaN	95.0	1	93.10	9.520000	
...	...	...	...	...	...	...	...	...	
18864	95002476	Manish Mittal	CSE	manish.mittal@gmail.com	73.0	0	0.00	8.370000	
18866	95002169	Aditi Mittal	CSE	aditi.mittal@gmail.com	73.0	0	0.00	5.440000	
18867	95000277	Vikas Kulshreshtha	CSE	vikas_kulshreshtha@outlook.com	97.0	0	0.00	7.405868	
18868	95001809	Neel Choudhary	CSE	neel.choudhary@gmail.com	70.0	1	71.73	9.140000	
18869	95000191	Snehal Garg	DS	snehal.garg@gmail.com	93.0	1	89.73	7.670000	

18000 rows × 13 columns

```
df1.value_counts("Placed") #we will calculate the no. students placed (1)or not placed(0)
```

Placed	count
0	9459
1	9411

dtype: int64

```
df1.isnull().sum()
```

	0
Enrollment_ID	0
Name	0
Department	0
Email	5261
Attendance	0
Mock_Status	0
Mock_Score	0
CGPA	0
Internships	0
Projects	0
Aptitude_Score	0
Communication_Score	0
Placed	0

dtype: int64

```
Placed_avg_student=df1.groupby("Department")["Placed"].mean()
print(Placed_avg_student*100)
```

```
Department
AIML    49.476791
CSE     49.455800
CSIT    49.639519
DS      50.065121
IT      50.712025
Name: Placed, dtype: float64
```

```
counts=df1.groupby("Department")["Placed"].value_counts()
print(counts)
```

```
Department  Placed
AIML        0         1883
            1         1844
CSE         0         1904
            1         1863
CSIT        0         1886
            1         1859
DS          1         1922
            0         1917
IT          1         1923
            0         1869
Name: count, dtype: int64
```

```
df1["Profile_Score"]=df1["Internships"]+df1["Projects"]
```

```
df1["Skill_Score"]=(df1["Aptitude_Score"]+df1["Communication_Score"])/2
```

```
df1.drop(["Enrollment_ID","Name","Email"],axis=1,inplace=True)
```

```
df1.groupby("Placed")[["CGPA","Aptitude_Score","Communication_Score"]].mean()
```

Placed	CGPA	Aptitude_Score	Communication_Score
0	7.409110	62.557174	62.863374
1	7.402609	62.908323	62.619958

```
df1["Placed"] = (
    (df1["CGPA"] > 7) &
    (df1["Aptitude_Score"] > 60) &
    (df1["Communication_Score"] > 60)
).astype(int) # here we set some condition to check that studets placed or not placed to improve the accuracy of model
```

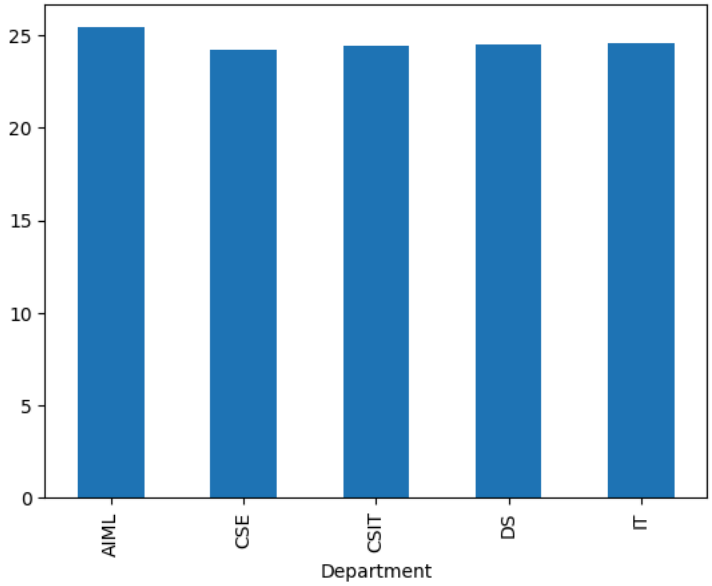
Name,Email,Enrollment_Id are not much relevant to our prediction so we can skip that part in email .

The data is been cleaned as there are no null values present in the details which are required for prediction .

EDA (Visualization)

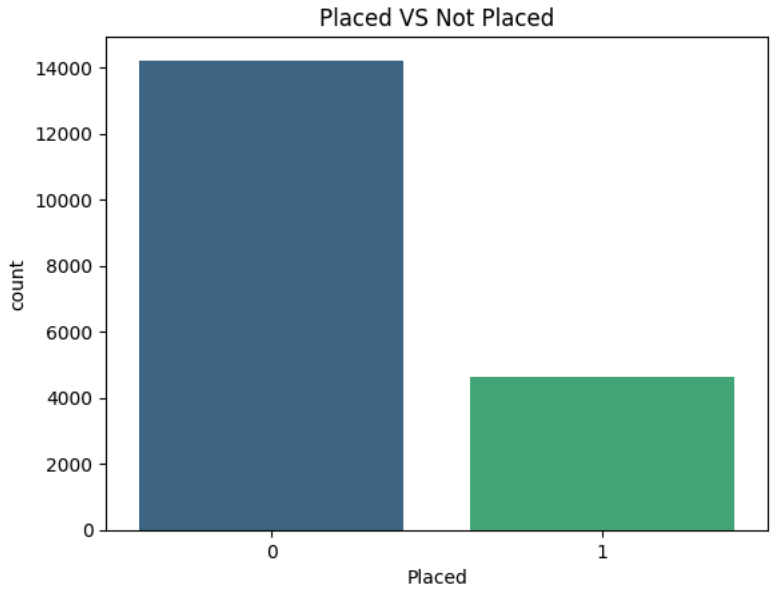
```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
placement_rate=df1.groupby("Department")["Placed"].mean()*100
placement_rate.plot(kind="bar")
plt.show()
```



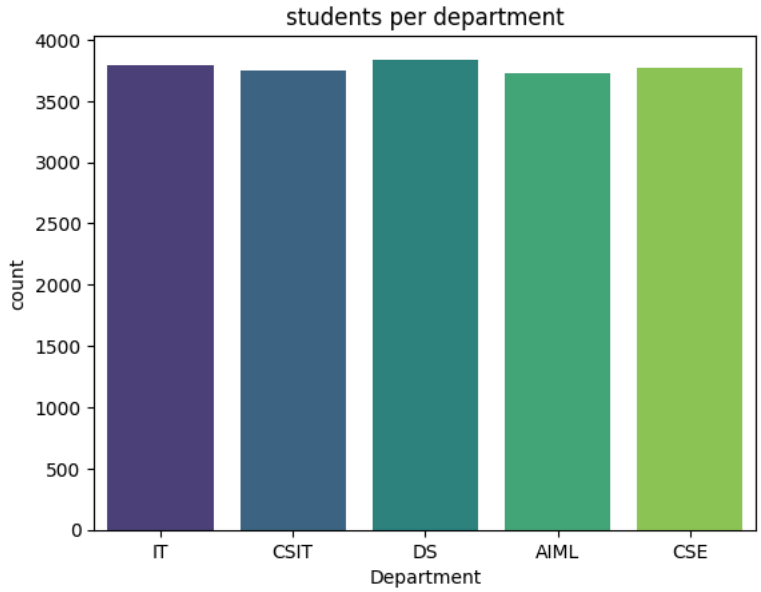

```
sns.countplot(x="Placed", data=df1,palette='viridis')
plt.title("Placed VS Not Placed")
plt.show()
```

```
/tmp/ipykernel_4752/1273134183.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

  sns.countplot(x="Placed", data=df1,palette='viridis')
```

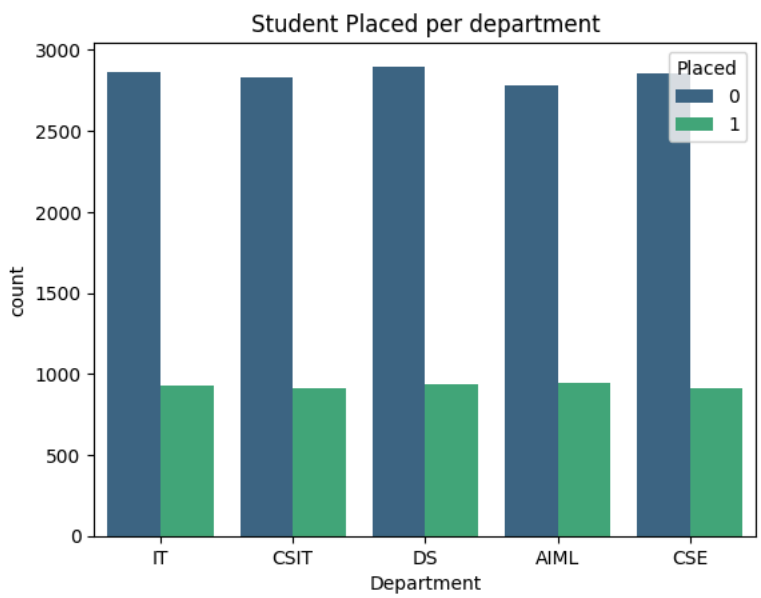



```
sns.countplot(x="Department",data=df1,palette='viridis')
plt.title("students per department")
plt.show()
```

```
/tmp/ipykernel_4752/3949723440.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

  sns.countplot(x="Department",data=df1,palette='viridis')
```



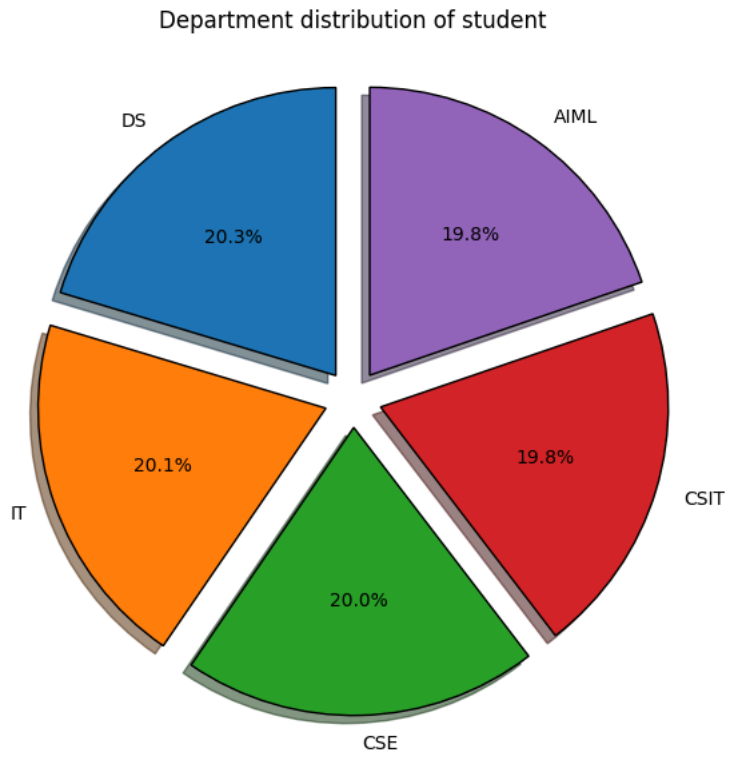

```
sns.countplot(x="Department",hue="Placed",data=df1,palette='viridis')
plt.title("Student Placed per department")
plt.show()
```



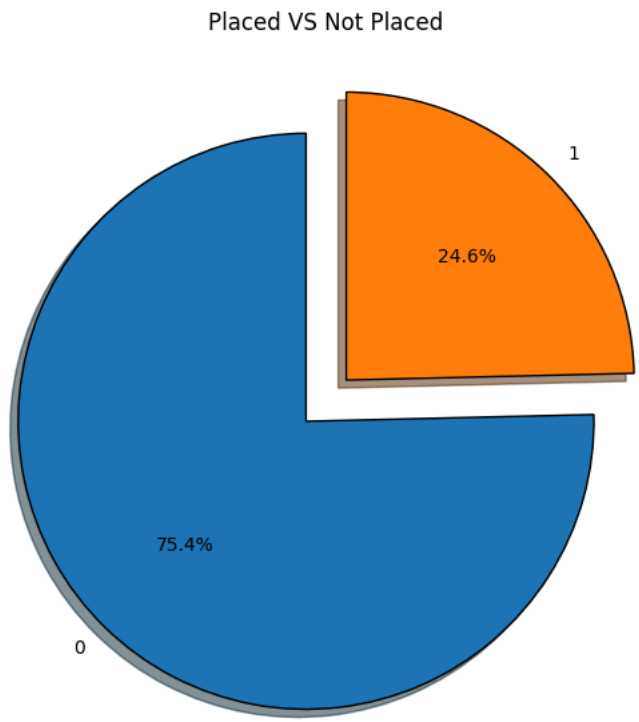
Pie - Chart

used for better understanding and visualiation of the data.

```
fig=plt.figure(figsize=(7,7))
explode=(0.1,0.1,0.1,0.1,0.1)
wp={"linewidth":1,"edgecolor":"black"}
tags=df1["Department"].value_counts()
tags.plot(kind="pie",autopct="%1.1f%%",shadow=True, startangle=90,wedgeprops=wp,explode=explode)
plt.title("Department distribution of student")
plt.ylabel("")   # removes extra label
plt.show()
```

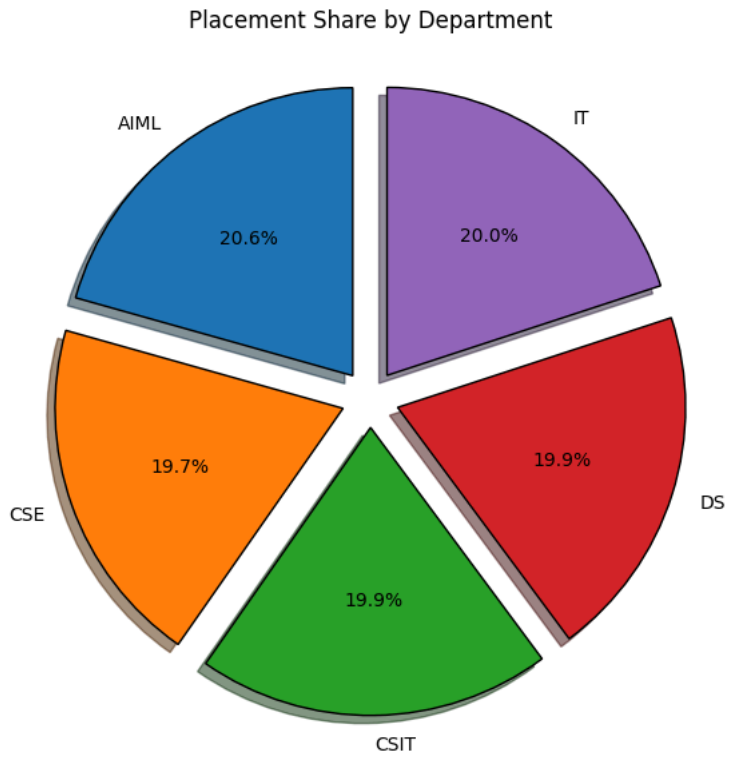



```
fig=plt.figure(figsize=(7,7))
explode=(0.1,0.1)
wp={"linewidth":1,"edgecolor":"black"}
tags=df1["Placed"].value_counts()
tags.plot(kind="pie",autopct="%1.1f%%",shadow=True, startangle=90,wedgeprops=wp,explode=explode)
plt.title("Placed VS Not Placed")
plt.ylabel("")   # removes extra label
plt.show()
```



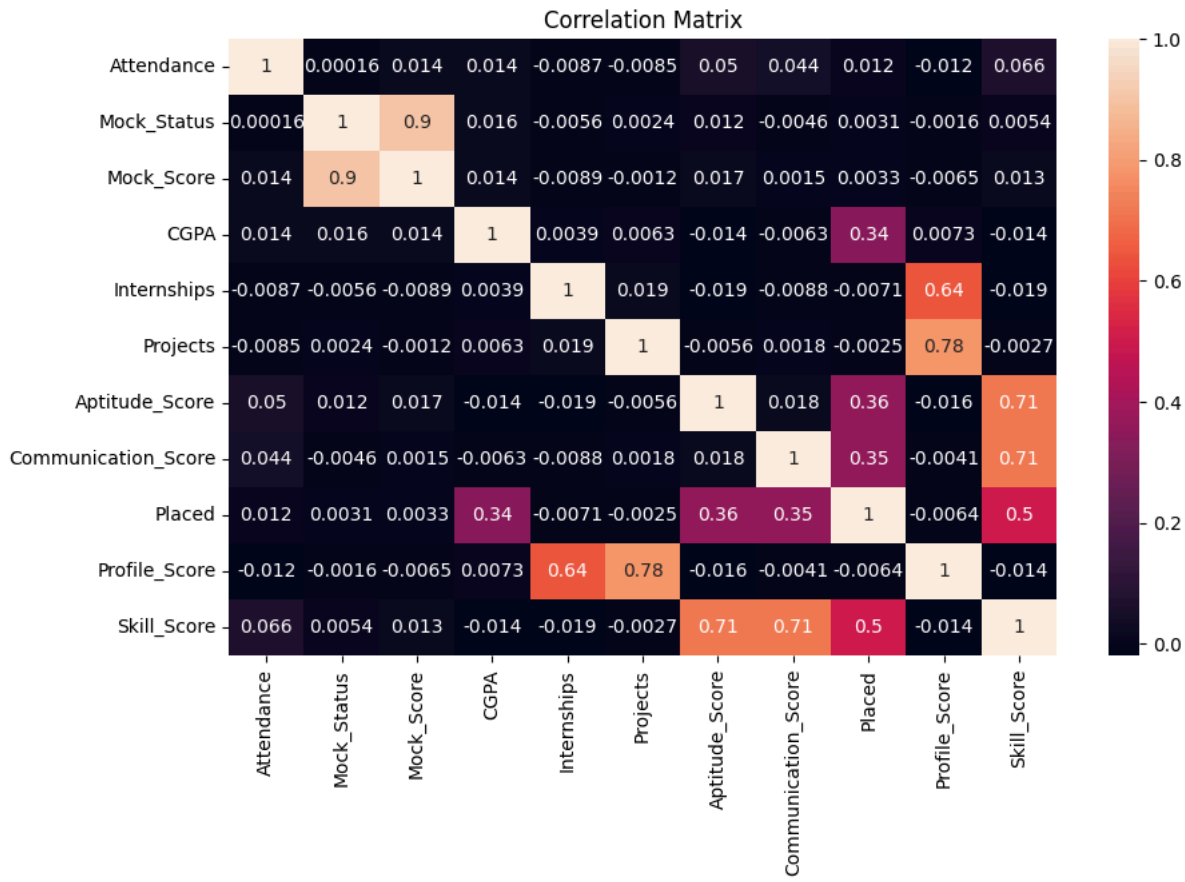

```
fig=plt.figure(figsize=(7,7))
explode=(0.1,0.1,0.1,0.1,0.1)
wp={"linewidth":1,"edgecolor":"black"}
```

```
placement_rate.plot(kind="pie", autopct="%1.1f%%",shadow=True, startangle=90,wedgeprops=wp,explode=explode)

plt.title("Placement Share by Department")
plt.ylabel("")
plt.show()
```



Heat-Map used for correleation matrix

```
corr = df1.corr(numeric_only=True)

plt.figure(figsize=(10,6))
sns.heatmap(corr, annot=True)
plt.title("Correlation Matrix")
plt.show()
```



Model Train and Testing

```
x=np.array(df1[["Mock_Score","CGPA","Aptitude_Score","Communication_Score","Internships","Projects"]])
y=np.array(df1[["Placed"]])
```

Below we have trained the model :

1. Logistic Regression
2. Decission Tree
3. Naive Bayes
4. Support Vector Machine

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score,classification_report
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```
from sklearn.linear_model import LogisticRegression
lr_model=LogisticRegression()
lr_model.fit(x_train,y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:1408: DataConversionWarning: A column-vector y was passed wh
  y = column_or_1d(y, warn=True)
▾ LogisticRegression
LogisticRegression()
```

```
lr_pred=lr_model.predict(x_test)
print("logisitic Regression \n")
print("accuracy:",accuracy_score(y_test,lr_pred))
```

```
logisitic Regression

accuracy: 0.8481717011128775
```

```
from sklearn.tree import DecisionTreeClassifier
dt_model=DecisionTreeClassifier()
dt_model.fit(x_train,y_train)
```

```
▾ DecisionTreeClassifier
DecisionTreeClassifier()
```

```
dt_pred=dt_model.predict(x_test)
print("Decision Tree Classifer \n")
print("accuracy:",accuracy_score(y_test,dt_pred))
```

```
Decision Tree Classifer

accuracy: 1.0
```

```
from sklearn.naive_bayes import MultinomialNB
nb_model=MultinomialNB()
nb_model.fit(x_train,y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:1408: DataConversionWarning: A column-vector y was passed wh
  y = column_or_1d(y, warn=True)
```

```
▾ MultinomialNB
MultinomialNB()
```

```
nb_predict=nb_model.predict(x_test)
print("Naive Bayes")
print("accuracy",accuracy_score(y_test,nb_predict))
```

```
Naive Bayes
accuracy 0.5105988341282459
```

```
from sklearn.svm import SVC
svm_model=SVC()
svm_model.fit(x_train,y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:1408: DataConversionWarning: A column-vector y was passed wh
  y = column_or_1d(y, warn=True)
```

```
▾ SVC
SVC()
```

```
svm_predict=svm_model.predict(x_test)
print("Support Vector Machine")
print("accuracy",accuracy_score(y_test,svm_predict))
```

```
Support Vector Machine
accuracy 0.8979862215156332
```

```
from sklearn.ensemble import RandomForestClassifier
rf_model=RandomForestClassifier()
rf_model.fit(x_train,y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/base.py:1389: DataConversionWarning: A column-vector y was passed when a 1d arra
  return fit_method(estimator, *args, **kwargs)
```

```
▾ RandomForestClassifier
RandomForestClassifier()
```

```
rf_predict=rf_model.predict(x_test)
print("random forest ")
print("accuracy:",accuracy_score(y_test,rf_predict))
```

```
random forest
accuracy: 1.0
```

```
features=np.array([[50,4,5,79,70,2,5]])
```

```
features = np.array([[50,4.5,75,70,2,5]])
print("Random Forest",rf_model.predict(features))
print("Naive Bayes",nb_model.predict(features))
print("Logistic Regression",lr_model.predict(features))
print("svm",svm_model.predict(features))
print("Decision Tree",dt_model.predict(features))
```

```
Random Forest [0]
Naive Bayes [0]
Logistic Regression [0]
svm [0]
Decision Tree [0]
```

```
df1.head(50)
```

	Department	Attendance	Mock_Status	Mock_Score	CGPA	Internships	Projects	Aptitude_Score	Communication_Score	Place
0	IT	42.0	0	0.00	7.200000	4.0	4.0	20.540000	83.310000	
1	CSIT	47.0	0	0.00	6.980000	2.0	1.0	25.330000	52.950000	
2	DS	87.0	0	0.00	7.405868	4.0	6.0	66.760000	22.200000	
3	DS	52.0	0	0.00	8.750000	1.0	1.0	70.980000	62.741976	
4	AIML	95.0	1	93.10	9.520000	1.0	2.0	33.470000	90.010000	
5	DS	73.0	1	59.89	8.070000	1.0	3.0	72.860000	62.741976	
6	IT	75.0	1	39.82	8.970000	2.0	4.0	67.560000	43.590000	
7	AIML	82.0	0	0.00	7.760000	0.0	6.0	42.830000	62.741976	
8	CSE	73.0	0	0.00	8.350000	4.0	1.0	84.490000	55.870000	
9	DS	73.0	0	0.00	6.520000	3.0	2.0	62.732302	96.250000	
10	DS	85.0	1	33.56	7.640000	2.0	6.0	23.610000	79.790000	
11	CSE	73.0	1	31.62	9.040000	1.0	6.0	29.980000	77.020000	
12	CSE	83.0	0	0.00	6.650000	0.0	1.0	32.280000	71.760000	
13	CSIT	93.0	0	0.00	5.300000	1.0	6.0	91.850000	62.300000	
14	CSIT	92.0	0	0.00	5.010000	2.0	6.0	41.750000	53.370000	
15	DS	79.0	0	0.00	9.790000	0.0	1.0	54.700000	55.320000	
16	IT	81.0	1	55.44	7.405868	1.0	2.0	39.550000	34.680000	
17	CSIT	42.0	1	60.09	7.360000	1.0	3.0	91.080000	62.741976	
18	IT	95.0	1	53.41	5.020000	4.0	5.0	31.260000	59.050000	